RIDENOW

Cloud-Native Ride-Sharing Backend

Project Proposal & Engineering Learning Plan

Primary Focus

Java • Spring Boot • Microservices • Concurrency • Distributed Systems • Redis • Messaging • Kubernetes • Amazon EKS

A deliberately scoped ride-sharing platform designed to provide realistic backend engineering problems while building practical cloud-native and Kubernetes skills.

Team Proposal • August 2026

1. Project Overview

RideNow is a simplified ride-sharing platform inspired by services such as Grab and Uber. Passengers request rides and drivers accept and complete them. The user experience stays intentionally simple; the engineering challenge is the backend underneath it.

The project evolves progressively: start with a working Spring Boot application, then introduce microservices, concurrency control, Redis, asynchronous messaging, event-driven communication, distributed transactions, real-time updates, Kubernetes, Amazon EKS, observability, CI/CD, load testing, and failure testing.

Scope principle: We are not reproducing a commercial ride-hailing platform. We are building a technically meaningful system where every advanced technology solves a concrete backend problem.

2. Project Goals

- Build a realistic Java/Spring Boot backend rather than another basic CRUD application.
- Understand microservice boundaries, independent deployment, and database ownership.
- Practice concurrency problems such as two drivers accepting the same ride simultaneously.
- Learn distributed-system patterns: asynchronous messaging, idempotency, eventual consistency, Saga/compensation, retries, and failure handling.
- Use Redis for caching, geospatial driver lookup, shared real-time state, and distributed coordination.
- Deploy containerized services to Kubernetes on Amazon EKS.
- Keep the Flutter mobile client lightweight so backend engineering remains the main focus.
- Measure and test the system under load and deliberately exercise failure scenarios.

3. What the User Actually Does

The product should be understandable without knowing the architecture. The following is the core passenger and driver experience.

Passenger journey

- Register or log in.
- Choose pickup and destination.
- Select a ride type and request a ride.
- Wait while the system searches for a nearby available driver.
- See the assigned driver's basic information and live location.
- Receive status updates when the driver arrives and the trip starts.
- Complete the trip and view the calculated fare.
- Pay using a simulated payment service.
- Rate the driver and view trip history.

Driver journey

- Log in as a driver.
- Switch between OFFLINE and AVAILABLE.
- Share current location while online.
- Receive a ride request and accept or decline it.
- Navigate to the passenger, mark ARRIVED, then START TRIP.

• Complete the trip and see earnings/history.

4. Core Features → Engineering Problems

Feature	User-facing behavior	Backend problem / topic
Authentication	Register, login, profile	JWT, Spring Security, hashing, authorization
Ride request	Choose destination and request	REST, validation, transactions, domain modeling
Driver availability	Go online/offline	State management, shared state, consistency
Driver matching	Find a nearby driver	Redis GEO, geospatial search, ranking
Accept ride	Driver taps Accept	Concurrency, race conditions, locks
Live tracking	Driver moves on passenger map	WebSockets, high-frequency updates, Redis
Payment	Pay after trip	Idempotency, retries, transaction boundaries
Notifications	Driver accepted/arrived/completed	SQS, workers, retries, DLQ
Trip events	Status changes trigger actions	EventBridge, loose coupling, eventual consistency
Failure recovery	Safe behavior during outages	Timeouts, retries, circuit breaker, compensation
History/earnings	Past trips and earnings	Pagination, indexing, analytics
Scaling	Remain responsive under load	HPA, load balancing, capacity planning

5. Target Architecture

The architecture evolves during development. The final target is independently deployable Spring Boot services running on EKS, with PostgreSQL for durable business data, Redis for fast/shared state, and AWS messaging services for asynchronous workflows.

```
Flutter Mobile → API entry point → Microservices
User • Driver • Trip • Matching • Payment • Notification Worker
PostgreSQL per service • ElastiCache/Redis • SQS • EventBridge • WebSocket layer
EKS → Deployments/Services/HPA → Observability + CI/CD
```

Target services

Service	Responsibility	Key concepts
User Service	Accounts, authentication, profiles	JWT, Spring Security, authorization
Driver Service	Driver state and location	State, Redis, WebSocket
Trip Service	Ride lifecycle and trip state	Transactions, state machine, concurrency
Matching Service	Find/rank nearby drivers	Redis GEO, algorithms, distributed state
Payment Service	Simulated payment processing	Idempotency, retries, Saga
Notification Worker	Process notification jobs	SQS, retries, DLQ, idempotent consumer
Analytics Service (optional)	Consume trip events	EventBridge, eventual consistency

6. Backend Knowledge Base

Each feature should be treated as a learning laboratory: implement the simplest correct version, identify its limitations, then improve it when a new engineering concept is introduced.

Area	Topics to learn
API Design	REST, DTOs, validation, HTTP status codes, pagination, API contracts

Area	Topics to learn
Spring Boot	Dependency injection, JPA/Hibernate, configuration, exception handling
Security	JWT, access/refresh tokens, password hashing, role-based authorization
Database	ACID, transactions, isolation, indexes, query plans, connection pools
Concurrency	Race conditions, pessimistic locking, optimistic locking, atomic updates
Redis	Cache-aside, TTL, GEO, atomic operations, distributed locks, Pub/Sub
Microservices	Boundaries, database-per-service, service communication, failure isolation
Messaging	SQS producers/consumers, visibility timeout, retries, DLQ, idempotency
Events	EventBridge, event contracts, loose coupling, eventual consistency
Distributed transactions	Saga pattern, compensation, workflow failure
Resilience	Timeouts, exponential backoff, circuit breaker, graceful degradation
Real-time	WebSockets, connection lifecycle, shared state, high-frequency updates
Testing	Unit, integration, Testcontainers, concurrency tests, load tests

7. Signature Problem: Concurrent Ride Acceptance

One ride can be visible to several nearby drivers. If two drivers press Accept at nearly the same time, the system must guarantee exactly one valid winner.

```
Incorrect approach: SELECT ride → check status → UPDATE ride. Two requests can both observe the same state before either update commits.
```

The team will implement and compare:

- **Pessimistic locking:** database row locking such as SELECT ... FOR UPDATE.
- **Optimistic locking:** a version field that allows only the request with the expected version to update.
- **Redis/distributed coordination:** investigate a distributed lock or atomic operation when coordination extends beyond one database.

Create a repeatable concurrency test that sends many simultaneous acceptance requests and proves that exactly one driver wins.

8. Important Distributed Flows

Driver matching: Trip requested → Matching Service queries nearby available drivers in Redis → candidates are ranked → offer is delivered → first valid acceptance wins.

Async notification: Trip event → SQS → Notification Worker → notification provider/simulation → success or retry → DLQ after repeated failure.

Payment and compensation: Trip completed → Payment Service → success/failure. Failure is handled through explicit state and compensation rather than assuming a transaction can span independent databases.

Live location: Driver GPS → WebSocket → backend → Redis/shared state → passenger WebSocket. Durable business information remains in PostgreSQL.

9. AWS & EKS Learning Plan

Service / platform	Purpose
Amazon EKS	Run and orchestrate Spring Boot microservices with Kubernetes
ECR	Store versioned Docker images

Service / platform	Purpose
ALB / Ingress	Route external HTTP traffic to Kubernetes services
RDS PostgreSQL	Managed relational storage for durable business data
ElastiCache for Redis	Shared cache, driver location, GEO, coordination
SQS	Async jobs, retries, worker decoupling, DLQ
EventBridge	Publish business events to independent consumers
CloudWatch	Logs, metrics, alarms and AWS operational visibility
Secrets Manager	Store credentials and sensitive configuration
IAM	Least-privilege access to AWS resources
VPC	Private networking, routing and security boundaries

10. Kubernetes Topics

Kubernetes topic	How it appears in RideNow
Cluster / Node	EKS infrastructure running workloads
Pod	Containerized service instance
Deployment	Replica management and rolling updates
Service	Stable internal networking
Ingress	HTTP routing to workloads
ConfigMap	Non-sensitive configuration
Secret	Sensitive configuration
HPA	Horizontal scaling based on metrics
Readiness/Liveness	Traffic safety and failed-container recovery
Namespaces	Environment/workload organization
Requests/Limits	CPU/memory scheduling and protection

11. Development Roadmap

Phase	Deliverable	Main learning
1	Spring Boot monolith + mobile MVP	REST, security, PostgreSQL, domain modeling
2	Trip lifecycle	Transactions, state machine, validation
3	Concurrency lab	Race conditions, pessimistic/optimistic locking
4	Redis + matching	Cache, GEO, shared state
5	Microservice split	Boundaries, database ownership, service communication
6	WebSocket tracking	Real-time communication, Redis Pub/Sub
7	SQS notification workers	Async processing, retries, DLQ, idempotency
8	Payment + Saga	Distributed transactions, compensation
9	EventBridge	Events, loose coupling, eventual consistency
10	Docker + EKS	Pods, Deployments, Services, Ingress
11	Production hardening	HPA, probes, resources, Secrets, IAM

Phase	Deliverable	Main learning
12	Observability + CI/CD	CloudWatch, metrics, logs, automated deployment
13	Load + failure testing	k6, scaling, resilience, incident analysis

12. Frontend Scope

Recommended stack: **Flutter + Dart**. Keep the mobile client deliberately lightweight because the team's main learning target is backend engineering and cloud-native deployment.

- Passenger: Login → Home/Map → Request Ride → Searching → Driver Assigned → Live Trip → Payment → Rating → History.
- Driver: Login → Driver Home → Go Online → Ride Request → Accept → Arriving → Start Trip → Complete → Earnings.
- Use REST for normal business operations and WebSockets for live updates.
- Avoid spending significant time on visual polish or advanced mobile-only features.

13. Suggested Team Responsibilities

Area	Typical responsibilities
Backend / Domain	Spring Boot services, API contracts, business rules, database design
Distributed Systems	Redis, SQS, EventBridge, idempotency, Saga, concurrency testing
Kubernetes / Platform	Docker, EKS, Deployments, Services, Ingress, HPA, Secrets
AWS / Infrastructure	VPC, RDS, ElastiCache, IAM, ECR, CloudWatch
Mobile	Flutter screens, API integration, authentication, WebSocket client
Testing / Observability	JUnit, Testcontainers, k6, dashboards, failure scenarios

14. Testing Strategy

- **Unit:** business rules, fare calculation, state transitions.
- **Integration:** PostgreSQL/Redis interactions and service boundaries.
- **Testcontainers:** reproduce real dependencies locally.
- **Concurrency:** hundreds of simultaneous ride-acceptance requests; verify one winner.
- **Load:** k6 for latency, throughput, error rate and scaling.
- **Failure:** kill Pods, stop workers, inject timeouts, duplicate messages, and make dependencies unavailable.

15. Definition of Done

Area	Evidence
Application	Passenger and driver can complete a full ride lifecycle
Microservices	Services deploy and scale independently
Concurrency	Concurrent test proves exactly one driver wins
Redis	Nearby-driver lookup and shared state work correctly
Messaging	SQS worker demonstrates retry and DLQ behavior
Idempotency	Duplicate attempts do not create duplicate business effects

Area	Evidence
Distributed workflow	Failure path demonstrates compensation/Saga behavior
Kubernetes	EKS workloads use probes, resources, rolling updates and HPA
Observability	Logs and metrics support investigation of failed requests
CI/CD	Changes can be tested, containerized, pushed and deployed automatically
Performance	Load-test results and scaling behavior are documented
Failure handling	Controlled failure scenarios have documented recovery behavior

16. Final Vision

From the user's perspective, RideNow is intentionally simple: request a ride, find a driver, track the trip, pay, and review. From the engineering perspective, the system becomes a practical laboratory for concurrency, service failure, asynchronous work, distributed state, consistency, scalability, observability, and deployment.

The project should be judged less by the number of screens and more by whether the team can explain **why each architectural decision exists, what failure it prevents, what trade-off it introduces, and how the system behaves under load or partial failure.**

Recommended starting point: Build the smallest end-to-end ride flow first. Do not begin with EKS or six microservices. Add complexity only when the previous version works and the next engineering problem gives us a reason to introduce it.